Project Onboarding & Execution Plan

Project: AI-Based Intelligent Policy Retrieval and Analysis System

Architecture: Retrieval-Augmented Generation (RAG)

1. Immediate Action Required (GitHub Access)

Before we start, I need to give you all access to the master code repository. 👉
EVERYONE: Direct message me your email address (the one linked to your GitHub account) on WhatsApp immediately. Once I add you, check your email, accept the invitation, and clone the repository to your laptop.

---

2. The Golden Rules of Our Project

Read this before you write a single line of code.

1. **Never commit to `main`:** All work must be done on your own branch. You must open a Pull Request (PR) to merge your code.
2. **Stick to the API Contract:** Frontend and Backend teams must adhere exactly to the JSON formats agreed upon below. Do not change variable names without a team meeting.
3. **Use the Exact Tech Stack:** Do not install random libraries. Use the exact versions listed in our `requirements.txt` and `package.json`.

---

3. Branch Naming Conventions (Version Control Rules)

To keep our repository organized, **every new branch must follow the `<type>/<short-description>` format.**

- **`feature/`** → When building a brand new part of the project (e.g., `feature/chat-interface`, `feature/pdf-chunking`).
- **`fix/`** → When fixing a bug in code that is already in the main branch (e.g., `fix/chromadb-crash`).
- **`docs/`** → When adding or updating documentation (e.g., `docs/add-srs-document`).
- **`refactor/`** → When rewriting existing code to make it cleaner, but not changing its behavior.

---

4. The API Contract (Frontend 🤝 Backend Agreement)

This defines exactly how the React UI talks to the Backend API. The Frontend Developer will use this to build the UI with "fake" data, while the Backend Developer builds the real logic.

Endpoint A: Ask a Question

- **URL:** `POST /api/v1/query`
- **What the Frontend Sends (JSON):**

  JSON

  ```
  {
    "question": "What is the maximum scholarship amount for IT
  students?",
    "inference_mode": "local"
  }
  ```

- **What the Backend Returns (JSON):**

  JSON

  ```
  {
    "status": "success",
    "data": {
      "answer": "The maximum scholarship amount is Rs. 50,000 per
  year.",
      "citations": [
        {
          "document_name": "UGC_Guidelines_2024.pdf",
          "page_number": 14,
          "clause": "Section 3.2"
        }
      ]
    }
  }
  ```

---

5. The "AI Prompting" Protocol (CRITICAL)

If you are using ChatGPT, Gemini, or Claude to help you write code, you must give the AI the complete context. Otherwise, it will generate code that breaks our project structure.

Step A: Upload the `SOFTWARE REQUIREMENTS SPECIFICATION [FINAL].pdf` document directly to your AI chat. **Step B:** Copy and paste the following prompt block into the AI before asking it to write any code. Fill in the bracketed `[ ]` information!

Copy and Paste this into your AI assistant:

"Act as a Senior Software Engineer. I am working on a 3rd-year engineering Mini Project called the 'AI-Based Intelligent Policy Retrieval and Analysis System'. I have attached the official Software Requirements Specification (SRS) document; please read it to understand our architecture, tech stack, and non-functional requirements.

We are working in a shared GitHub repository. Here is the link to our repo: https://github.com/OmIngle82/Smart-Policy-Retrieval-System.git. Assume the following base folder structure: `/frontend`, `/backend`, `/ai_engine`, `/data_pipeline`, and `/docs`.

My Role: [INSERT YOUR ROLE, e.g., Data / Ingestion Specialist] **My Current Task:** [EXPLAIN YOUR SPECIFIC TASK HERE]

Rules for your response:

1. **Step-by-Step Guidance:** Explain exactly what I need to do step-by-step. Tell me the exact file name I need to create and exactly which folder to put it in.
2. **Tech Stack Adherence:** Only provide code that strictly aligns with the technology stack mentioned in the SRS document.
3. **Ask Before Assuming:** If my request lacks necessary context—such as missing database schemas, API JSON contracts, or specific library versions—DO NOT guess or invent them. Stop and ask me to provide that information before you write the code.
4. **Explain the Code:** Add brief, clear comments to the code so I can explain the logic during my college viva presentation."

---

6. Role Assignments & Step-by-Step Instructions

Role 1: Data / Ingestion Specialist

Your Job: Convert messy PDF files into clean, searchable mathematics (Vectors).

- **Step 1:** Download 3-5 sample government policy PDFs.
- **Step 2:** Write Python scripts to extract text from PDFs, chunk the data, and generate embeddings.
- **Step 3:** Store embeddings in the Vector Database (ChromaDB or FAISS).
- **End Goal:** Provide the AI Engineer with a working Vector Database file that they can query.

Role 2: AI / NLP Engineer

Your Job: Build the "Brain" of the project using RAG.

- **Step 1:** Install Ollama locally and download the `llama3` model. Set up a Google Gemini API key for Cloud mode.
- **Step 2:** Implement the Hybrid Search algorithm (Vector + Keyword).
- **Step 3:** Integrate Ollama and Gemini API, establishing the Dual Mode inference toggle.
- **Step 4:** Ensure your code outputs "Grounded Citations" linking responses to the exact page/clause of source PDFs.
- **End Goal:** A Python function that takes a text string (question) and returns a text string (answer) + citations.

Role 3: Backend Developer

Your Job: Build the bridge between the Frontend UI and the AI Engine, and manage the database.

- **Step 1:** Design the MySQL database schema for user authentication and document metadata.
- **Step 2:** Develop the backend APIs (Spring Boot/FastAPI) to handle basic routing.
- **Step 3:** Create the `/api/v1/query` endpoint that receives JSON from the Frontend, sends it to the AI Engine, and returns the JSON answer.
- **Step 4:** Enforce Role-Based Access Control (RBAC) filtering on all search queries.
- **End Goal:** A fully running local server that accepts HTTP requests and returns correct JSON responses.

Role 4: Frontend Developer

Your Job: Build the visual interface using React.js and PDF.js.

- **Step 1:** Create the base React app and build a Chat Interface.
- **Step 2:** Build an Admin page with a file upload button for new PDFs.
- **Step 3:** Use "Mock Data" (fake JSON responses) to build the UI immediately.
- **Step 4:** Develop the split-screen PDF viewer that highlights the exact text snippet when a citation is clicked.
- **End Goal:** A responsive web interface that successfully sends queries to the Backend APIs.

Role 5: QA, Testing & Integrator

Your Job: Protect the `main` branch, test the code, and piece it all together.

- **Step 1:** Ensure everyone has cloned the GitHub repository and knows how to make branches.
- **Step 2:** Write unit tests (JUnit or PyTest) to validate the core backend logic.
- **Step 3:** Code Review: When team members submit a PR, review their code before merging it into `main`.
- **Step 4:** Build the logic that toggles the system between "Local Mode" (Ollama) and "Cloud Mode" (Gemini).
- **End Goal:** A stable, bug-free master repository that can run completely offline or on the cloud.